

Asignación de Memoria Dinámica

01

Concepto

Fundamentos de la memoria dinámica en C

02

Características

Variables locales y mapa de memoria

03

Utilización

Funciones malloc, calloc, realloc y free

04

Gestión

Ejemplos prácticos y buenas prácticas

LIC. JONATHAN G. PÉCORA

Mapa de Memoria de un Programa

Para comprender la memoria dinámica, primero es necesario entender cómo se organiza la memoria de un programa en ejecución. Cada programa tiene un espacio de memoria estructurado en distintas regiones con propósitos específicos.

Variables Locales

Almacenadas en la pila (stack). Se crean al entrar en una función y se destruyen al salir.

Variables de main

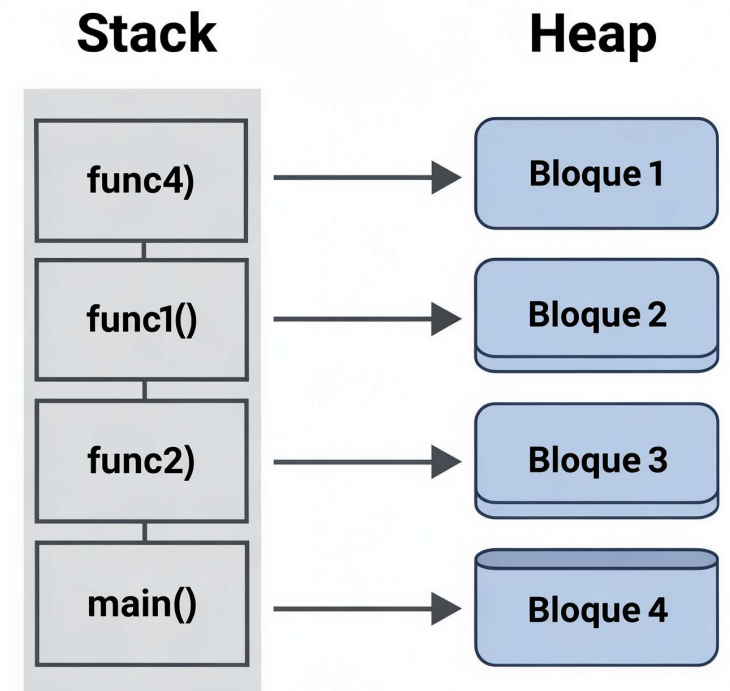
Son variables locales de la función principal. Existen durante toda la ejecución del programa.

Constantes

Valores fijos definidos en tiempo de compilación. Residen en el segmento de datos o de texto.

Memoria Dinámica

*Región del **Heap**. Se asigna y libera explícitamente durante la ejecución del programa.*



Características de las Variables Locales

Las variables locales son aquellas declaradas dentro de una función.

Su comportamiento está determinado por dos propiedades fundamentales que todo programador debe conocer.

Alcance (Scope)

*Las variables locales son **visibles únicamente dentro del bloque o función donde se declaran**. No pueden ser accedidas desde otras funciones. Su tamaño es **fijo y conocido en tiempo de compilación**.*

- *Solo accesibles dentro de su bloque { }*
- *No comparten espacio con otras funciones*
- *El compilador conoce el tamaño exacto*

Ciclo de Vida

*Las variables locales **nacen al entrar en el bloque** donde se declaran y **mueren al salir** de él. La memoria que ocupan en la pila se libera automáticamente.*

- *Se crean en tiempo de ejecución al llamar la función*
- *Se destruyen automáticamente al retornar*
- *No persisten entre llamadas sucesivas*



A diferencia de la memoria dinámica, no requieren gestión manual por parte del programador.

Memoria Dinámica: El Concepto

La memoria dinámica es un concepto fundamental en programación que permite **asignar y gestionar memoria en tiempo de ejecución**, a diferencia de las variables locales cuyo tamaño se fija en tiempo de compilación.

¿Qué problema resuelve?

En muchos programas no sabemos de antemano cuánta memoria necesitaremos. Por ejemplo, al leer datos del usuario, procesar ficheros de tamaño variable o construir estructuras de datos que crecen dinámicamente.

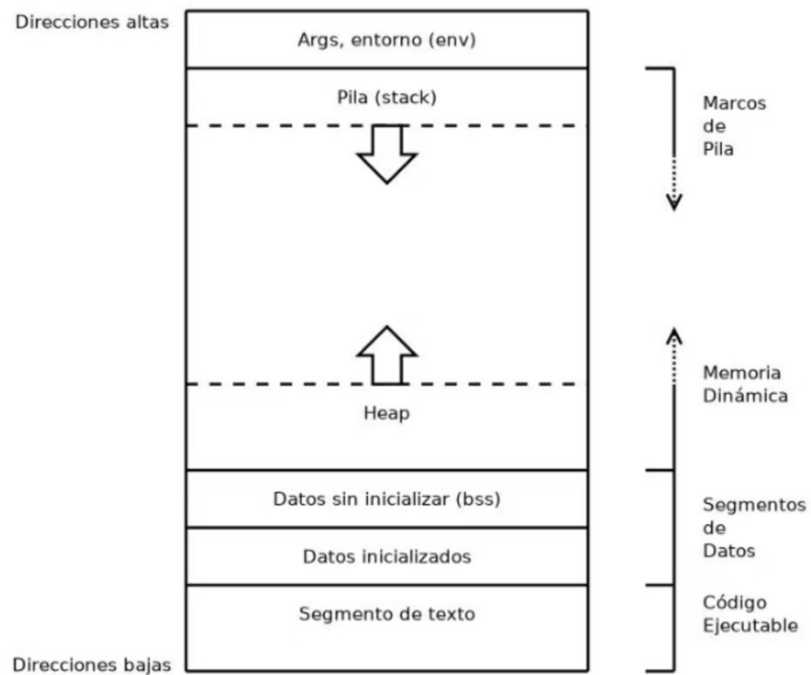
- Tamaño determinado en ejecución, no en compilación
- Permite estructuras de datos flexibles y escalables
- Región de memoria utilizada: el **HEAP**

Comparativa

Característica	Stack	Heap
Tamaño	Fijo	Variable
Gestión	Automática	Manual
Momento	Compilación	Ejecución
Velocidad	Rápida	Más lenta

Mapa de Memoria de un Proceso

Todo proceso en ejecución dispone de un espacio de direcciones de memoria organizado en regiones bien definidas. Comprender esta estructura es esencial para gestionar correctamente la memoria dinámica.



Args / Entorno (env)

Argumentos de línea de comandos y variables de entorno del sistema operativo.

Pila (Stack)

Marcos de pila para cada llamada de función: variables locales e información de retorno. Crece hacia direcciones bajas.

Heap

Memoria dinámica asignada con malloc/calloc. Crece hacia direcciones altas.

BSS / Datos inicializados

Variables globales y estáticas, con o sin valor inicial explícito.

Segmento de texto

Instrucciones del programa en código máquina. Solo lectura.

Funciones de Gestión de Memoria Dinámica

La biblioteca estándar de C (`<stdlib.h>`) proporciona cuatro funciones esenciales para gestionar la memoria dinámica en el Heap. Conocerlas en profundidad es imprescindible para cualquier programador en C.



`malloc()`

Memory Allocation. Reserva un bloque de memoria del tamaño indicado en bytes. Devuelve un puntero `void*` al inicio del bloque, o `NULL` si falla.



`calloc()`

Contiguous Allocation. Reserva memoria para un array de n elementos, inicializando todos los bytes a cero. Más seguro que `malloc` para arrays.



`realloc()`

Reallocate Memory. Cambia el tamaño de un bloque previamente asignado. Puede mover el bloque a una nueva dirección si es necesario.

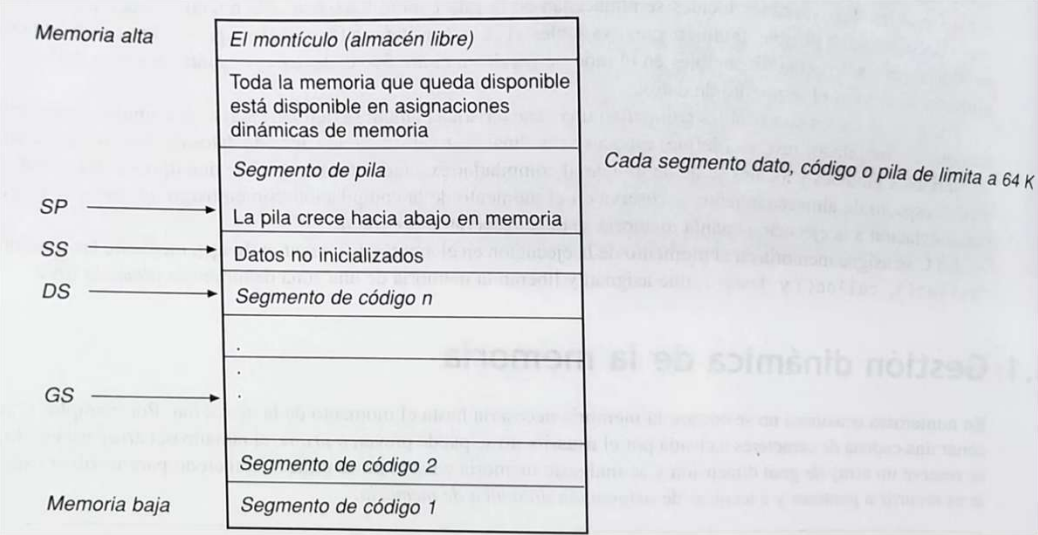


`free()`

*Libera un bloque de memoria previamente asignado con `malloc`, `calloc` o `realloc`. No liberar memoria produce **fugas de memoria**.*

Segmentos de Memoria en Detalle

El mapa de memoria de un proceso en arquitectura x86 incluye registros de segmento que apuntan a las distintas regiones.
Entender estos punteros es clave para depurar programas de bajo nivel.



SP – Stack Pointer

*Apunta a la cima actual de la pila.
Se actualiza en cada push y pop.*

SS – Stack Segment

Registro que define la base del segmento de pila en memoria.

DS – Data Segment

Apunta al segmento de datos donde residen las variables globales y estáticas.

GS – General Segment

Registro de propósito general para segmentos adicionales de código o datos.

❏ *Cada segmento (dato, código o pila) tiene un límite de 64 KB en arquitecturas de 16 bits.*

La Función `malloc()`: ¿Por qué devuelve `void*`?

`malloc()` reserva un bloque contiguo de memoria en el Heap del tamaño especificado en bytes y devuelve un puntero al inicio de dicho bloque. La forma es:

```
void* malloc(size_t tamaño_en_bytes);
```

¿Por qué devuelve `void*`?

`malloc` no sabe qué tipo de dato se va a almacenar en el bloque reservado. Por eso devuelve un puntero genérico `void*`, que puede apuntar a cualquier tipo. El programador debe realizar un **cast explícito** al tipo deseado.

- Devuelve `NULL` si no hay memoria disponible
- El tamaño se especifica siempre en **bytes**
- Usar `sizeof(tipo)` para portabilidad

Uso correcto con cast

```
int *p;  
p = (int*) malloc(N * sizeof(int));  
if (p == NULL) {  
    // Error: sin memoria  
    return 1;  
}
```

⚠ Si `malloc` devuelve `NULL`, nunca se referencia el puntero. Siempre verificar antes de usar la memoria asignada.

Uso Correcto de `malloc()` con Cast

Una vez comprendido por qué `malloc` devuelve `void*`, el siguiente paso es aplicar correctamente el **cast de tipo** para que el puntero quede tipado y se pueda usar de forma segura con aritmética de punteros.

✅ ¡Ahora Sí! Con el cast explícito, el compilador sabe cómo interpretar la dirección devuelta por `malloc`.

```
int *b;  
b = (int*) malloc(N * sizeof(int));
```

→ Declarar el puntero

`int *b;` — Se declara un puntero al tipo que queremos almacenar en el Heap.

→ Aplicar el cast

`(int*)` — Convierte el `void*` genérico a un puntero a entero, permitiendo operar sobre los datos correctamente.

→ Llamar a `malloc` con el tamaño correcto

`malloc(N * sizeof(int))` — Se solicitan `N` bloques del tamaño de un entero. `sizeof` garantiza portabilidad entre plataformas.

→ Solo devuelve la dirección de inicio

`malloc` únicamente retorna la dirección del primer byte del bloque. El tamaño del bloque no se almacena en el puntero.

Ejemplo Práctico: Asignación con `malloc()`

El siguiente fragmento ilustra el patrón básico de uso de `malloc`: reservar un array de enteros en el Heap, verificar el resultado, usar la memoria y liberarla al finalizar.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    int *array;

    printf("Ingrese la cantidad de enteros: "); // Cantidad de elementos
    scanf("%d", &n);

    array = (int *) malloc(n * sizeof(int)); // Reserva de memoria en el heap

    if (array == NULL) {
        printf("Error: no se pudo reservar memoria.\n");
        return 1;
    } // Verificación de malloc

    for (int i = 0; i < n; i++) {
        printf("Ingrese el valor %d: ", i + 1);
        scanf("%d", &array[i]);
    } // Uso de la memoria (cargar valores)

    printf("\nValores ingresados:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", array[i]);
    } // Mostrar valores

    printf("\n");

    free(array); // Liberar memoria

    array = NULL; // Buena práctica: evitar puntero colgante

    return 0;
}
```

Pasos del ejemplo

1. Declarar el puntero del tipo adecuado
2. Llamar a `malloc(N * sizeof(tipo))`
3. Verificar que el puntero no sea `NULL`
4. Usar el bloque de memoria asignado
5. Liberar con `free()` al terminar

Puntos clave

- El puntero `b` actúa como un array en el Heap
- Se puede acceder con `b[i]` o `*(b+i)`
- La memoria persiste hasta llamar a `free(b)`
- No olvidar incluir `#include <stdlib.h>`



Olvidar llamar a `free()` provoca una **fuga de memoria** (memory leak).
El bloque permanece ocupado aunque el puntero ya no sea accesible.

Ejemplo Completo: Parte 1 – Inicialización y `malloc`

Este ejemplo muestra cómo construir dinámicamente una cadena de caracteres. Se comienza con una capacidad inicial de 10 caracteres y se crece según sea necesario usando `realloc`.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    char *cadena = NULL; // Inicializa puntero a char a NULL
    char character;
    int longitud = 0;     // Longitud actual de la cadena
    int capacidad = 10;  // Capacidad inicial del bloque

    cadena = (char *)malloc(capacidad * sizeof(char));

    if (cadena == NULL) {
        printf("Error: No se pudo asignar memoria dinámica.\n");
        return 1;
    }

    printf("Ingresa caracteres (presiona Enter para finalizar):\n");
    // Lee caracteres hasta que se presiona Enter ('\n')
```

 Se inicializa el puntero a `NULL` y se verifica el resultado de `malloc` antes de usar la memoria. Esta es la práctica correcta y segura.

Ejemplo Completo: Parte 2 – **realloc** Dinámico

Cuando la longitud de la cadena supera la capacidad reservada, se duplica el tamaño del bloque con **realloc**.

Esta estrategia de duplicación amortiza el costo de las reasignaciones.

```
while ((caracter = getchar()) != '\n') {
    longitud++;
    if (longitud >= capacidad) {
        // Si la longitud supera la capacidad actual,
        // se duplica la memoria reservada
        capacidad *= 2;
        cadena = (char *)realloc(cadena, capacidad * sizeof(char));

        if (cadena == NULL) {
            printf("Error: No se pudo asignar más memoria.\n");
            return 1;
        }
    }
    // Agrega el nuevo carácter a la cadena
    cadena[longitud - 1] = caracter;
}
```

 **Precaución:** Si **realloc** falla y devuelve **NULL**, el puntero original se pierde. Es recomendable usar una variable temporal para guardar el resultado antes de asignarlo.

Ejemplo Completo: Parte 3 – Terminación y `free()`

Tras leer todos los caracteres, se añade el carácter nulo para formar una cadena válida en C, se imprime el resultado y se libera la memoria dinámica con `free()`.

```
// Agrega el carácter nulo para formar una cadena válida en C
cadena[longitud] = '\0';

printf("La cadena ingresada es: %s\n", cadena);

// Liberar la memoria dinámica cuando ya no se necesite
free(cadena);

return 0;
}
```

`'\0'` es obligatorio

Sin el carácter nulo al final, funciones como `printf("%s, ...)` no sabrían dónde termina la cadena, causando comportamiento indefinido.

`free()` siempre

Toda memoria asignada con `malloc/realloc` debe liberarse con `free()`. No hacerlo produce fugas de memoria acumulativas.

Puntero inválido

Tras llamar a `free(cadena)`, el puntero queda inválido. Asignarlo a `NULL` es una buena práctica para evitar accesos accidentales.



Consultas

¿Tienen preguntas sobre los temas vistos en esta clase? Este es el momento para resolver dudas, repasar conceptos y asegurarnos de que todos los fundamentos estén bien comprendidos antes de avanzar al siguiente módulo.

"La mejor pregunta es la que todavía no se hizo."

Lic. Jonathan G. Pécora - UTN · Instituto Nacional Superior del Profesorado Técnico